

gods_eye_v1_approach_4 — Async Coroutine Swarm with Shared Memory Coordination (Predecessor to PDF)

Title

gods_eye_v1_approach_4: Autonomous Coroutine Agents with Shared Memory Coordination, DAG-as-Data Planning, and Explicit Resilience Chains

Design Period: Conceptual Iteration 4

Status: Deprecated — Superseded by HackAgent v2 PDF (Final Architecture)

Predecessor: Approach 3 (LangGraph State Machine)

Problem Statement

Approach 3 revealed that the fundamental issue with graph-based orchestration is that the execution topology must be known before the LLM starts reasoning. For a penetration testing agent operating against unknown targets, this assumption fails immediately — the attack surface is discovered dynamically, and the task structure must be able to restructure itself at runtime in response to discoveries.

The correct reframe coming out of Approach 3 is: **the task graph should be data, not code**. If the task decomposition lives in a mutable data structure that the Planner can rewrite, the execution topology is no longer constrained by a graph compiled at design time.

Approach 4 implements this insight: the orchestration framework is eliminated as the coordination primitive, replaced by a shared `state.json` file that acts as the live task DAG. Agents are pure asyncio coroutines that react to state changes — they do not inherit a framework's lifecycle. The Planner writes and rewrites the task DAG in `state.json`; the Executor reads ready tasks and executes them; the Analyst writes findings to Qdrant and Mem0 and routes structured summaries back to the Planner.

This is architecturally the closest design to the PDF. The five-agent model is present, the shared memory coordination model is in place, the approval gate uses `asyncio.Event`, the resilience chain exists. The gap between Approach 4 and the PDF is in the **specific isolation rules, information flow boundaries, and the Reporter's architectural position** — details that appear minor but have significant impact on system correctness and parallelism guarantees.

Proposed Architecture

Overview

Five agent classes, each running as an independent asyncio coroutine:

1. **SupervisorAgent** — Plans, dispatches tasks, and replans on failure. Owns `state.json` writes for the `tasks` section.
2. **HackerAgent** — Autonomous execution coroutine per dispatched task. Proposes commands, waits for approval, executes, sends output to Analyst.
3. **AnalystAgent** — Receives raw output from Hacker, extracts findings, writes to Qdrant and Mem0, sends structured summary to Supervisor.
4. **ReporterAgent** — Handles user I/O. Surfaces command proposals for approval. Reads `state.json` for status queries.
5. **SearcherAgent** — Called by HackerAgent (when stuck) or SupervisorAgent (before replanning). Queries local ExploitDB mirror and synthesizes intel.

The agents are launched as `asyncio.create_task()` coroutines on a shared event loop. They coordinate through:

- `state.json` on disk (read/write via a per-file asyncio lock)
- `Mem0` entity graph (shared client)
- `Qdrant` vector store (standalone server, concurrent clients)
- Direct async function calls for: Supervisor→Hacker dispatch, Hacker→Analyst output routing, Analyst→Supervisor finding routing, Hacker→Searcher intel requests, Supervisor→Searcher intel requests

Component Breakdown

SupervisorAgent

Uses LangChain's `create_structured_chat_agent` with a set of planning and state-management tools. The Supervisor's LLM is prompted with the current `state.json` content, the current Mem0 entity set, and a top-10 Qdrant semantic query against "prior context for this target" at session start.

Tools available to Supervisor:

- `plan(goal: str) → TaskDAG` — Decomposes goal into task list with `depends_on` chains. Writes to `state.json`.
- `replan(failure_context: dict) → TaskDAGPatch` — Modifies subset of task DAG in response to Analyst findings.
- `dispatch(task_id: str)` — Sends task to a HackerAgent coroutine with full context payload.
- `set_nuance(key: str, value: str)` — Writes key-value fact to `state.json["nuances"]`.
- `skip(task_id: str, reason: str)` — Marks task as skipped.
- `abort(reason: str)` — Terminates goal with documented reason.
- `request_intel(query: str) → IntelResult` — Calls SearcherAgent synchronously before replan.

Behavior: Supervisor is not poll-driven. It is triggered by: (1) initial goal receipt from Reporter, and (2) Analyst routing a finding summary. Between triggers, it is idle.

HackerAgent

A fully autonomous coroutine that receives a task dispatch from Supervisor and runs an internal reasoning loop (up to 30 iterations). Each iteration:

1. Build LLM prompt: task description + goal + current nuances from dispatch payload + relevant Mem0 entities + top-5 Qdrant similar findings.
2. LLM proposes a command and reasoning.
3. Route proposed command + reasoning to ReporterAgent via async call.
4. Wait on `asyncio.Event` (set by Reporter on user approval/denial).
5. If approved: `subprocess.run()` the command. Collect full stdout/stderr.
6. If denied: Inject denial + user reason into LLM conversation. Loop.
7. Send raw output to AnalystAgent via async call.
8. If task complete: Call `task_complete(summary, findings)`.
9. If stuck after N retries: Call `request_intel(query)` on SearcherAgent, inject result, retry N more times.
10. If stuck after N+N retries: Call `signal_stuck(reason)`.

Failure handling: Non-zero exit code injects error into LLM conversation. HackerAgent adapts approach autonomously before escalating.

AnalystAgent

Receives raw output asynchronously from HackerAgent after each command. Does not wait for task completion — processes mid-task outputs in real time.

Processing pipeline per output:

1. LLM-structured extraction (Pydantic schema) → `List[FindingSpec]`.
2. Nuance inference: parse port states, service versions, OS fingerprints, credentials, WAF indicators.
3. Deduplication check: query Qdrant for cosine similarity > 0.95 before writing.
4. Write unique findings to Qdrant with `project_id`, `doc_type`, `severity`, `confidence_score` payload.
5. Write typed entities to Mem0 (Host, Service, Vulnerability, Credential, Network).
6. Route structured summary to Supervisor: `{task_id, status, findings, nuances, failure_reason}`.

ReporterAgent

Always-running coroutine on a separate asyncio task. Responsibilities:

- Accepts user goal input, writes to `state.json["goal"]`, signals Supervisor.

- Surfaces `propose_command` requests from HackerAgent to user, sets approval `asyncio.Event`.
- Answers user status queries by reading `state.json` + Mem0 + Qdrant.
- Displays findings in real time as Analyst emits them.
- Generates final report on goal completion.

SearcherAgent

On-demand coroutine. Uses `searchsploit` CLI (ExploitDB local mirror, `exploitdb` package) plus a LangChain `LLMChain` for synthesis. Provides three query types: CVE lookup, exploit search by service/version, technique synthesis. Returns structured `IntelResult` objects.

ExploitDB via searchsploit: The `searchsploit` binary is called via `subprocess.run` with `--json` flag for structured output. Results include exploit title, file path (local copy of PoC), CVE IDs, and platform. The LLM synthesizes these results into actionable tool flags and payload commands.

Libraries & Tools

asyncio (Python stdlib) All agents are `asyncio.Task` coroutines on a shared event loop. `asyncio.Event` is used for per-command approval gates — each HackerAgent instance creates its own `asyncio.Event` per proposed command, blocking only that Hacker's coroutine while other coroutines continue. This is the key advantage over LangGraph's `interrupt_before` which blocks the entire runner.

Limitations: asyncio's cooperative multitasking model requires all agent coroutines to `await` at appropriate points. CPU-bound operations (LLM inference, large output parsing) are blocking unless explicitly offloaded to a thread pool via `asyncio.to_thread`. At this iteration, LLM calls are not thread-offloaded, causing brief event loop stalls during inference.

LangChain (langchain, langchain-community) Used for agent tool definitions, structured output parsing (Pydantic schemas), and LLM wrappers. LangChain's `StructuredOutputParser` is used in AnalystAgent for finding extraction. LangChain's `create_structured_chat_agent` is used in SupervisorAgent for tool-calling orchestration.

Limitations: LangChain's structured output parser occasionally fails on LLM outputs that do not strictly conform to the schema — requiring fallback parsing logic. The `create_structured_chat_agent` function uses a prompt template that injects all tool schemas on every call, adding ~2000 tokens of overhead per Supervisor LLM invocation.

Qdrant (qdrant-client, standalone Docker) Vector store with `project_id` isolation and multi-field filtering. Used by AnalystAgent for finding storage and by all agents for semantic retrieval. Qdrant collections are organized by `doc_type`: `finding`, `output`, `intel`, `code`. The `qdrant-client` async client (`AsyncQdrantClient`) is used to avoid blocking the event loop during Qdrant operations.

Limitations: This iteration uses a single Qdrant collection per `(doc_type)` across all engagements. Without `(project_id)` filtering applied consistently, cross-engagement retrieval pollution is possible. The design specifies `(project_id)` as a required payload filter, but there is no enforcement mechanism — a query that omits the `(project_id)` filter silently returns results from all engagements.

Mem0 (mem0ai) Entity graph memory. Used by AnalystAgent for typed entity writes and by SupervisorAgent, HackerAgent, and ReporterAgent for reads. Mem0's async client is used.

Limitations: At this iteration, Mem0 entity writes are unbatched — each entity is written in a separate API call. For an nmap scan that discovers 50 open ports, AnalystAgent makes 50+ sequential Mem0 write calls, blocking the Analyst coroutine for several seconds. This is the most significant latency bottleneck in the architecture.

searchsploit / ExploitDB (local mirror) `(searchsploit)` binary wrapped in `(subprocess.run)` with `(--json)`. A full local ExploitDB mirror is maintained at `(/opt/exploitedb)`. SearcherAgent calls `(searchsploit -j <query>)` and parses the JSON results. PoC files are read from the local mirror and included in the `(IntelResult)` payload.

Selection rationale: Local ExploitDB mirror eliminates network dependency for exploit research. `(searchsploit --json)` provides structured output (no HTML parsing). The full PoC file content is available locally, allowing SearcherAgent to include actual exploit code in its intel synthesis.

subprocess (Python stdlib) Used in HackerAgent for command execution. `(subprocess.run)` with `(capture_output=True)`, no timeout at this iteration (timeout is managed by the HackerAgent's iteration budget, not by subprocess). Commands are executed on the host directly. A persistent Docker container execution path is available but not the default.

pymetasploit3 Metasploit RPC integration, carried forward from Approach 3. Used for post-exploitation tasks requiring Metasploit modules. Session IDs are stored as nuances in `(state.json)` for cross-task session reuse.

aiofiles Async file I/O library for `(state.json)` reads and writes. All agents that access `(state.json)` use `(aiofiles)` with a shared `(asyncio.Lock)` to prevent concurrent write corruption. Read operations are unlocked (multiple concurrent readers allowed); write operations acquire the lock exclusively.

Failure Points

1. HackerAgent Writes Raw Outputs Directly to Qdrant, Bypassing Analyst Schema

In this iteration, HackerAgent writes raw command outputs to Qdrant `((doc_type: "output"))` immediately after execution, before AnalystAgent processes them. The intent is to make raw outputs semantically searchable for future tasks. However, this creates a **dual-write race condition**: HackerAgent writes the raw output at $t=0$, AnalystAgent writes the structured finding derived from that output at $t=5$. Both documents exist in Qdrant with different `(doc_type)` values. When HackerAgent queries Qdrant for "similar past findings," it retrieves

both the raw output and the structured finding, causing the LLM to process the same information twice with different levels of structure, leading to confused context and duplicated reasoning.

More critically: the raw output document has no `severity` or structured metadata, so severity-filtered queries miss it. The structured finding has no `raw_output_hash` linkage to its source document. The two documents are logically related but architecturally disconnected.

2. SupervisorAgent Receives Finding Summaries But Not Per-Command Nuances

AnalystAgent routes a structured summary to Supervisor after task completion (success or failure). However, nuances inferred from mid-task command outputs are not routed to Supervisor in real time — they are batched into the task-completion summary. This means:

If HackerAgent is 10 commands into a 30-command task and discovers "WAF detected" from command 3's output, AnalystAgent infers `waf_detected: true` but does not route this to Supervisor until the task completes. Meanwhile, commands 4-10 proceed without WAF bypass techniques, because the Supervisor has not yet injected `waf_detected: true` into the nuances section of `state.json`.

The delayed nuance propagation causes **stale nuance injection**: HackerAgent builds its LLM prompt with the nuances from dispatch time, not from real-time discoveries. The architecture notionally supports real-time nuance injection (AnalystAgent can call `set_nuance` on Supervisor at any point) but does not implement it for mid-task outputs — only for task-final summaries.

3. ReporterAgent Blocks Execution During Report Generation

The ReporterAgent runs on a shared asyncio event loop with all other agents. Report generation (the `write_final_report()` call at goal completion) involves multiple sequential LLM calls (executive summary, per-finding descriptions, recommendations). During report generation, the ReporterAgent's coroutine holds the event loop for the duration of each LLM call (because LLM calls are not thread-offloaded).

More significantly: ReporterAgent's `surface_question` (surfacing HackerAgent decision questions to the user) and `notify_finding` (displaying live findings) are synchronous within the Reporter coroutine. If the Reporter is rendering a complex status display when a new `notify_finding` call arrives, the notification is queued — it does not display in real time. The architecture describes the Reporter as "always running in parallel, never blocking" but the shared event loop means any CPU-bound operation in the Reporter blocks all other agents for its duration.

4. Supervisor→Hacker Dispatch Passes Context as a Serialized Payload, Not via Memory

When SupervisorAgent dispatches a task to HackerAgent, it passes task context (goal, nuances, relevant Mem0 entities, Qdrant findings) as a serialized dict payload. This means:

- The context is a snapshot taken at dispatch time. If nuances change between dispatch and the Hacker's first command, the Hacker does not see the updates.

- If the same task is re-dispatched after a replan (with updated strategy), the old dispatch payload may still be in the Hacker's LLM conversation history if the Hacker was mid-execution.
- Large context payloads (many prior findings, many Mem0 entities) inflate the Hacker's initial LLM prompt by thousands of tokens on every dispatch, regardless of relevance to the specific task.

The PDF resolves this by having HackerAgent query Mem0 and Qdrant independently before each task, rather than receiving a pre-assembled context payload. This separates context retrieval from task dispatch and ensures the Hacker always queries the latest memory state.

5. `asyncio.Lock` on `state.json` Is a Serialization Bottleneck

All `state.json` writes are serialized through a single `asyncio.Lock`. In a scenario where three HackerAgent instances complete tasks nearly simultaneously, their three AnalystAgent coroutines each attempt to route nuances and task summaries to Supervisor, which each attempt to acquire the `state.json` write lock. The lock queue serializes all three writes. Total latency for all three state updates: 3× the per-write latency.

More problematically: the SupervisorAgent, which needs to read `state.json` to identify newly ready tasks after each completion, also acquires the read path. In this iteration, reads are not write-locked (multiple concurrent readers allowed), but the writer lock acquisition pattern uses a standard `asyncio.Lock` rather than a `asyncio.Semaphore` or `asyncio.RWLock` — meaning a single writer blocks all readers for the lock duration. The PDF's architecture avoids this by assigning strict write ownership (Supervisor writes tasks, Reporter writes goal/status only) and using atomic section updates rather than whole-file locks.

6. SearcherAgent Synthesis Is LLM-Only — No Structured ExploitDB Field Extraction

SearcherAgent queries ExploitDB via `searchsploit --json` and then passes the raw JSON result to an LLM for synthesis. The LLM extracts tool flags, payload commands, and bypass techniques from unstructured PoC text. This works for common, well-documented exploits. For novel or complex PoCs (multi-stage exploits, exploits requiring specific environment setup), the LLM synthesis is unreliable — it may miss preconditions, invent flags, or confuse similar-named exploits.

The architecture lacks structured ExploitDB field extraction: CVSS score, affected platform, CVE-to-exploit mapping, and PoC verification status are not parsed from ExploitDB's structured data and are not included in the `IntelResult` schema. Supervisor and Hacker receive synthesized prose, not structured exploit metadata.

7. HackerAgent "Stuck" Signal Goes Directly to AnalystAgent — Not via Output Processing

When HackerAgent exhausts its retry budget and calls `signal_stuck(reason)`, this signal is routed directly to AnalystAgent without passing through the standard output processing pipeline. This means:

- No partial findings are extracted from the commands that were attempted before the stuck signal.
- The failure context sent to Supervisor is the HackerAgent's self-assessed failure reason, not AnalystAgent's structured analysis of the attempted commands' outputs.
- Nuances discoverable from failed command outputs (e.g., "permission denied" → infer `(needs_privesc: true)`) are not extracted if the command sequence ended in a stuck signal rather than a natural task completion.

The PDF explicitly routes all command outputs through Analyst even on stuck escalations, ensuring no intelligence is lost from failed attempts.

8. No Explicit Isolation of Who Reads/Writes What in state.json

In Approach 4, the state.json write rules are documented but not enforced. HackerAgent is supposed to not write to state.json, but there is no structural barrier preventing it from doing so. An LLM reasoning step in the HackerAgent could use the `(set_nuance)` tool (intended for Supervisor only) if the LangChain tool definitions are not scoped per-agent. This violates the isolation model and can cause state corruption — HackerAgent writing a nuance that Supervisor then reads as authoritative, without the nuance having passed through Analyst's structured extraction.

The PDF enforces this through explicit tool-to-agent mapping: each agent is initialized with a disjoint tool set, and no agent has access to tools outside its domain. The access matrix `((state.json Mem0 Qdrant))` is enforced at initialization time, not by documentation convention.

Comparison to Approach 3

Approach 4 makes the decisive shift from framework-orchestrated to memory-coordinated:

Problem in Approach 3	How Approach 4 Addresses It
Static graph topology	Task DAG as mutable data in <code>state.json</code> ; Supervisor rewrites at runtime
<code>interrupt_before</code> serial approval bottleneck	Per-Hacker <code>asyncio.Event</code> ; each Hacker blocks only on its own approval
LangGraph step-level blocking on long commands	Free-running asyncio coroutines; no graph step boundaries
AnalystNode sequential bottleneck	AnalystAgent is a separate coroutine processing outputs concurrently
Replanner has no external intelligence	SupervisorAgent calls SearcherAgent before replan decisions
No dedicated user interface agent	ReporterAgent is a dedicated always-on coroutine
State object unbounded growth	<code>state.json</code> + Mem0 + Qdrant as separate external stores; state.json contains only task metadata
Metasploit session state external	Session ID stored as nuance in <code>state.json</code> ; visible to Supervisor for dependency modeling

Approach 4 is the first iteration where the architecture is genuinely capable of completing a full penetration testing engagement end-to-end. The failure modes that remain are correctness and efficiency issues, not fundamental capability gaps.

Why Approach 4 Is the Best Among Approaches 1-4, and Why the PDF Still Supersedes It

Among the four approaches, Approach 4 solves the most failure modes:

- Approach 1's context collapse → solved by Mem0 + Qdrant external memory.
- Approach 1's no approval gate → solved by `asyncio.Event` per command.
- Approach 2's message ordering inconsistencies → solved by shared state coordination.
- Approach 2's agent crash silent halt → solved by explicit resilience chain with `signal_stuck` escalation.
- Approach 3's static topology trap → solved by DAG-as-data in `state.json`.
- Approach 3's serial approval bottleneck → solved by per-Hacker `asyncio.Event`.
- Approach 3's replanner knowledge isolation → solved by Supervisor→Searcher intel requests.

However, Approach 4 has eight specific failure modes that the PDF resolves. The gap between Approach 4 and the PDF is not in overall capability — both can complete engagements — but in the **correctness of information flow, enforcement of isolation rules, and elimination of subtle race conditions** that cause incorrect behavior in edge cases.

Why This Is the Predecessor to the PDF (Approach 5)

Approach 4 is the immediate predecessor because it has the same five-agent structure, the same three-store memory architecture, the same `asyncio.Event` approval mechanism, the same DAG-as-data planning model, and the same resilience chain as the PDF. The PDF is a refinement of Approach 4, not a redesign.

The specific gaps the PDF closes over Approach 4:

What the PDF Fixes Over This Approach

Failure in Approach 4	PDF Solution
HackerAgent raw writes to Qdrant bypass Analyst schema	Strict write rules: only Analyst writes to Mem0; Hacker writes raw outputs to Qdrant only (<code>doc_type: "output"</code>), Analyst writes structured findings separately; PDF's access matrix clarifies this formally
Delayed nuance propagation (mid-task nuances not routed to Supervisor)	Analyst routes nuances to Supervisor after every command output, not just at task completion; Supervisor immediately updates <code>state.json["nuances"]</code> and re-injects into subsequent Hacker dispatches
ReporterAgent blocking on shared event loop	PDF explicitly separates Reporter to a dedicated coroutine with no CPU-bound operations in the critical path; LLM calls in Reporter are async with <code>asyncio.to_thread</code> offloading
Dispatch payload as snapshot — stale context	HackerAgent queries Mem0 and Qdrant independently at task start and before each command; context is always current-state, not dispatch-time snapshot
<code>state.json</code> write lock bottleneck	Strict section ownership (Supervisor writes tasks, Reporter writes goal/status); atomic section updates using <code>aiofiles</code> seek + write, not whole-file rewrite
SearcherAgent LLM-only synthesis	PDF's SearcherAgent structures ExploitDB results into typed <code>IntelResult</code> with CVSS, CVE mapping, platform, and PoC file path; synthesis is structured, not prose-only

Failure in Approach 4

PDF Solution

`signal_stuck` bypasses
Analyst output processing

PDF routes all outputs through Analyst regardless of task outcome;
stuck signal includes all command outputs attempted, not just the
failure reason

No structural enforcement of
agent tool isolation

PDF enforces per-agent tool sets at initialization; access matrix
(state.json / Mem0 / Qdrant R/W per agent) is enforced structurally, not
by documentation convention
